B_Leakage_PSD_Welch

April 21, 2026

1 Project 1: Spectral tools in practice

This notebook continues the first spectra notebook and focuses on three technical ideas:

1. the discrete FFT frequency grid,
2. spectral leakage and the effect of windowing,
3. the difference between an amplitude spectrum and a power spectral density (PSD).

Throughout this notebook, we keep the default sampling setup fixed to

$$N = 1000, \qquad f_s = 100 \text{ Hz}$$

so that you can transfer intuition from one widget to the next more easily.

[1]:
```
import numpy as np
import matplotlib.pyplot as plt
import ipywidgets as widgets
from IPython.display import display

try:
    get_ipython().run_line_magic("matplotlib", "widget")
except Exception:
    pass

from spectral_tools_helpers import (
    setup_notebook_style,
    float_slider,
    int_slider,
    dropdown,
    update_fft_grid_plot,
    update_fft_detune_plot,
    update_leakage_window_plot,
    update_psd_plot,
    update_spectrogram_plot,
    make_stft_demo_signal,

)
```

```
setup_notebook_style()
```

1.1 Widget 1. FFT rigor: the discrete frequency grid

For a signal with N samples and sampling frequency f_s, the FFT evaluates the spectrum only on the discrete grid

$$f_k = k\frac{f_s}{N}, \qquad k = 0, 1, 2, \ldots$$

with frequency spacing

$$\Delta f = \frac{f_s}{N}.$$

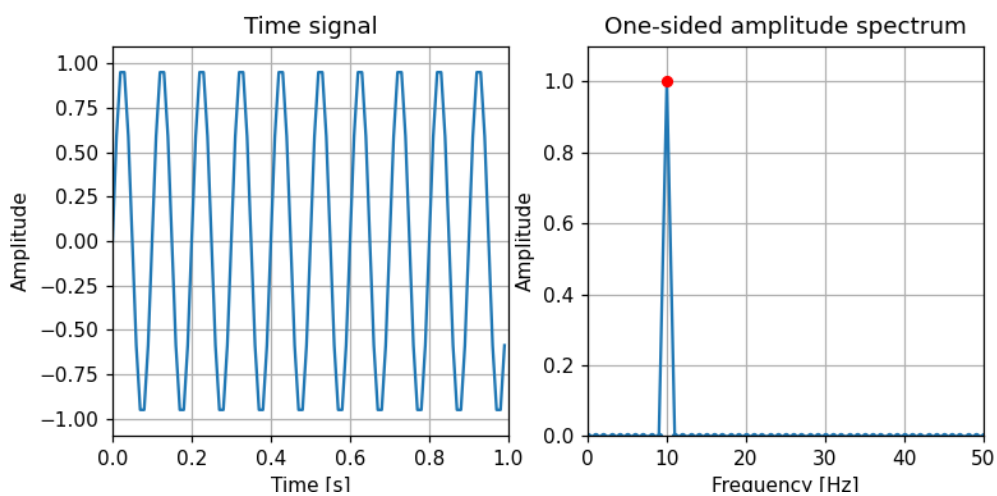
A sinusoid whose frequency is exactly on this grid can be represented very cleanly by the FFT.

Mini exercise. Vary the bin index k. Compute f_0 from $f_0 = k\Delta f$ and verify that the spectral peak stays concentrated in one bin.

[2]:
```
# Widget 1: signal exactly on the FFT grid
fs_fixed = 100
N_fixed = 100
A0_fixed = 1.0

bin_w = int_slider(value=10, min=1, max=100, step=1, description="bin index k")
phase_w = float_slider(value=0.0, min=0.0, max=360.0, step=5.0,␣
 ↪description="phase [deg]")
info_fft_grid = widgets.HTML()

fig_fft_grid, axes_fft_grid = plt.subplots(1, 2, figsize=(8, 3.6))
t_line_grid, = axes_fft_grid[0].plot([], [], lw=1.5)
s_line_grid, = axes_fft_grid[1].plot([], [], "o-", ms=2.5)
peak_marker_grid, = axes_fft_grid[1].plot([], [], "ro", ms=5)
ax_t = axes_fft_grid[0]
ax_s = axes_fft_grid[1]

ax_t.set_xlim(0, N_fixed / fs_fixed)
ax_t.set_ylim(-1.1*A0_fixed, 1.1*A0_fixed)
ax_s.set_xlim(0, min(fs_fixed/2, 50))
ax_s.set_ylim(0, 1.1*A0_fixed)
ax_t.set_title("Time signal")
ax_t.set_xlabel("Time [s]")
ax_t.set_ylabel("Amplitude")
ax_s.set_title("One-sided amplitude spectrum")
ax_s.set_xlabel("Frequency [Hz]")
ax_s.set_ylabel("Amplitude")
```

```
def _update_fft_grid(change=None):
    update_fft_grid_plot(
        t_line_grid, s_line_grid, peak_marker_grid,
        axes_fft_grid[0], axes_fft_grid[1], info_fft_grid,
        fs=fs_fixed, N=N_fixed, k=bin_w.value, phase_deg=phase_w.value,␣
 ↪A0=A0_fixed,
    )
    fig_fft_grid.canvas.draw_idle()

for w in (bin_w, phase_w):
    w.observe(_update_fft_grid, names="value")
_update_fft_grid()

display(widgets.VBox([widgets.HBox([bin_w, phase_w]), info_fft_grid,␣
 ↪fig_fft_grid.canvas]))
```

```
VBox(children=(HBox(children=(IntSlider(value=10, continuous_update=False,␣
 ↪description='bin index k', layout=L…
```



1.2 Widget 2: What if the signal is between FFT bins?

Real signals are usually not located exactly on the FFT grid. A convenient way to describe this is

$$f_0 = (k + \delta)\Delta f,$$

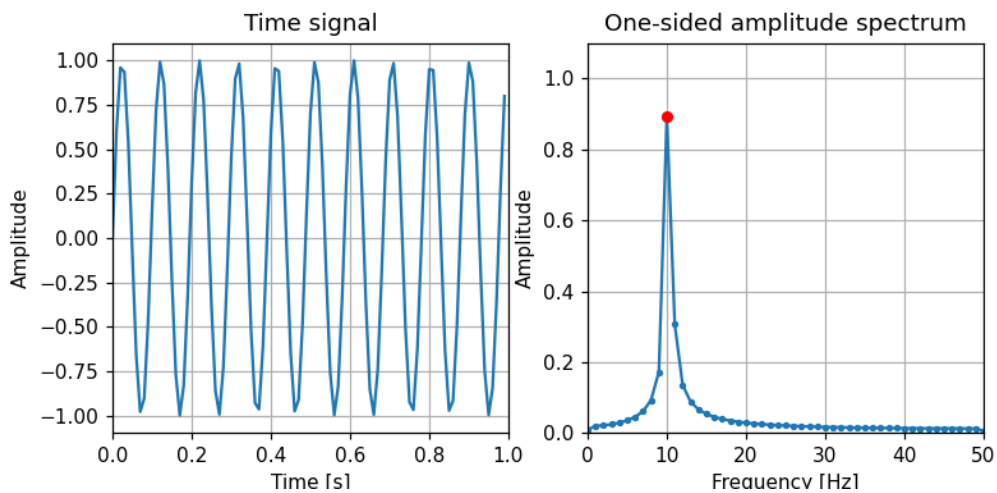
where k is the nearest bin index and δ is a fractional offset from that bin.

If $\delta \neq 0$, the energy is no longer concentrated in a single FFT bin. This is the transition to spectral leakage.

Mini exercise. Keep k fixed and vary the offset δ. At what values of δ is the peak most concentrated? What changes in the time signal, and what changes only in the spectrum?

[3]:
```
# Widget 2: off-grid frequency as a transition to leakage
bin_d_w = int_slider(value=10, min=1, max=200, step=1,
                     description="bin index k")
detune_w = float_slider(value=0.25, min=-0.5, max=0.5, step=0.05,
                        description="offset delta")
info_fft_detune = widgets.HTML()

fig_fft_detune, axes_fft_detune = plt.subplots(1, 2, figsize=(8, 3.6))
t_line_detune, = axes_fft_detune[0].plot([], [], lw=1.5)
s_line_detune, = axes_fft_detune[1].plot([], [], "o-", ms=2.5)
peak_marker_detune, = axes_fft_detune[1].plot([], [], "ro", ms=5)

ax_t = axes_fft_detune[0]
ax_s = axes_fft_detune[1]
ax_t.set_xlim(0, N_fixed / fs_fixed)
ax_t.set_ylim(-1.1, 1.1)
ax_t.set_title("Time signal")
ax_t.set_xlabel("Time [s]")
ax_t.set_ylabel("Amplitude")

ax_s.set_xlim(0, fs_fixed / 2)
ax_s.set_ylim(0, 1.1)
ax_s.set_title("One-sided amplitude spectrum")
ax_s.set_xlabel("Frequency [Hz]")
ax_s.set_ylabel("Amplitude")


def _update_fft_detune(change=None):
    update_fft_detune_plot(
        t_line_detune, s_line_detune, peak_marker_detune,
        axes_fft_detune[0], axes_fft_detune[1], info_fft_detune,
        fs=fs_fixed, N=N_fixed, k=bin_d_w.value, delta=detune_w.value, A0=1.0
    )
    axes_fft_detune[0].set_ylim(-1.1, 1.1)
    axes_fft_detune[1].set_ylim(0, 1.1)
    fig_fft_detune.canvas.draw_idle()

for w in (bin_d_w, detune_w):
    w.observe(_update_fft_detune, names="value")
_update_fft_detune()

display(widgets.VBox([widgets.HBox([bin_d_w, detune_w]), info_fft_detune,␣
 ↪fig_fft_detune.canvas]))
```

```
VBox(children=(HBox(children=(IntSlider(value=10, continuous_update=False,␣
↪description='bin index k', layout=L…
```



1.3 Widget 3. Spectral leakage and windowing

Because the FFT sees only a finite segment, it implicitly assumes that this segment is repeated periodically. If the signal does not match smoothly at the boundaries, the spectrum spreads over many bins.

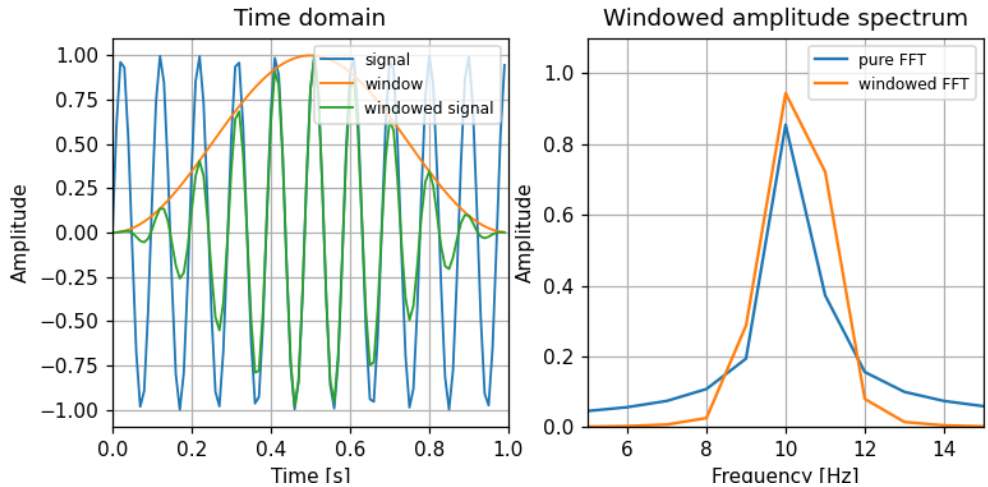
Windowing reduces the sharp discontinuity at the edges. The trade-off is that the main peak becomes broader.

Mini exercise. Vary only f_0 between 9 and 11 Hz. Compare the rectangular and Hann windows. Which one gives the sharper peak? Which one suppresses distant sidelobes more strongly?

[4]:
```
# Widget 3: spectral leakage and windowing
f0_l_w = float_slider(value=10.3, min=9.0, max=11.0, step=0.1, description="f0␣
↪[Hz]")
window_l_w = widgets.Dropdown(
    options=["rectangular", "hann", "hamming", "blackman", "tukey", "triang"],
    value="hann",
    description="window",
    layout=widgets.Layout(width="260px")
)
info_leak = widgets.HTML()

fig_leak, axes_leak = plt.subplots(1, 2, figsize=(8, 3.6))
raw_line, = axes_leak[0].plot([], [], lw=1.2, label="signal")
win_line, = axes_leak[0].plot([], [], lw=1.2, label="window")
xw_line, = axes_leak[0].plot([], [], lw=1.2, label="windowed signal")
specwin_line, = axes_leak[1].plot([], [], lw=1.5, label="pure FFT")
```

```python
spec_line, = axes_leak[1].plot([], [], lw=1.5, label="windowed FFT")

ax_left = axes_leak[0]
ax_left.set_xlim(0, N_fixed/fs_fixed)
ax_left.set_ylim(-1.1, 1.1)
ax_left.set_title("Time domain")
ax_left.set_xlabel("Time [s]")
ax_left.set_ylabel("Amplitude")
ax_left.legend(loc="upper right", fontsize=8)

ax_right = axes_leak[1]
ax_right.set_xlim(5, 15)
ax_right.set_ylim(0, 1.1)
ax_right.set_title("Windowed amplitude spectrum")
ax_right.set_xlabel("Frequency [Hz]")
ax_right.set_ylabel("Amplitude")
ax_right.legend(loc="upper right", fontsize=8)

def _update_leak(change=None):
    update_leakage_window_plot(
        raw_line, win_line, xw_line, spec_line, specwin_line,
        axes_leak[0], axes_leak[1], info_leak,
        fs=fs_fixed, N=N_fixed, f0=f0_l_w.value, window_name=window_l_w.value
    )
    axes_leak[0].set_ylim(-1.1, 1.1)
    axes_leak[1].set_ylim(0, 1.1)
    fig_leak.canvas.draw_idle()

for w in (f0_l_w, window_l_w):
    w.observe(_update_leak, names="value")
_update_leak()

display(widgets.VBox([widgets.HBox([f0_l_w, window_l_w]), info_leak, fig_leak.
 ↪canvas]))
```

```
VBox(children=(HBox(children=(FloatSlider(value=10.3, continuous_update=False,␣
 ↪description='f0 [Hz]', layout=L…
```



1.3.1 What to notice

A rectangular window keeps the narrowest main peak, but it tends to produce stronger sidelobes. Smoother windows such as Hann or Blackman reduce sidelobes, but they broaden the main lobe.

That is the central trade-off of windowing: - **better suppression of leakage far away from the peak**, versus - **poorer frequency resolution close to the peak**.

1.4 Widget 4. Amplitude spectrum vs power spectral density (PSD)

These two plots answer different questions:

- The **amplitude spectrum** asks: how large is the sinusoidal content at each frequency?
- The **PSD** asks: how much signal power is present per unit bandwidth?

For broadband noise, the PSD is often the more meaningful engineering quantity.

Mini exercise. Keep the noise level fixed and vary f_0. Then compare how the tone appears in the amplitude spectrum and in the PSD. Why is a logarithmic y-axis useful here? Change the window function and compare the resulting signal.

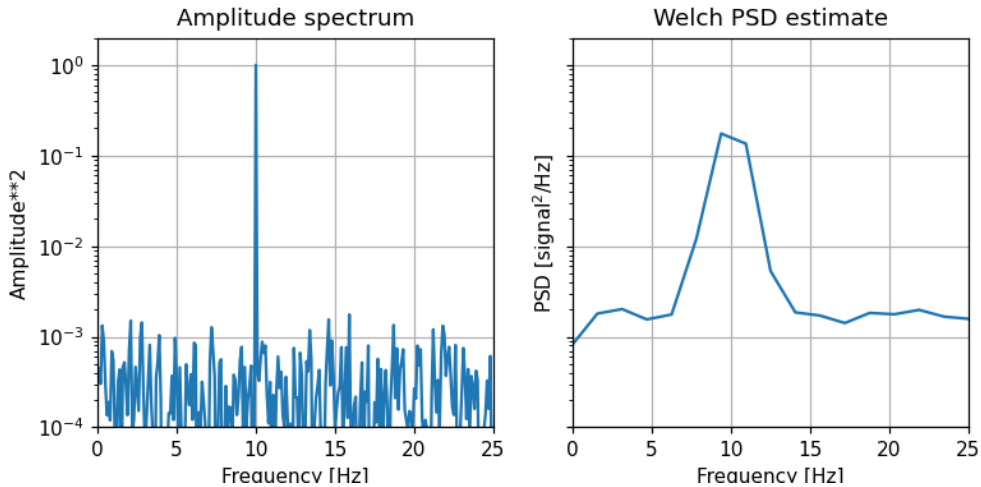
Key idea: Bias–variance trade-off in PSD estimation Welch averaging (small nperseg) reduces variance and produces a smoother spectrum, but broadens spectral peaks (lower frequency resolution). Using longer segments (large nperseg) improves resolution, but increases variance (noisier spectrum).

[5]:
```
# Widget 4: amplitude spectrum vs PSD
N_fixed = 100

print(fs_fixed/N_fixed)

N_w = dropdown(options=[100, 500, 1000], value=1000, description="N samples")
```

```
f0_p_w = float_slider(value=10.0, min=9.0, max=11.0, step=0.1, description="f0␣
↪[Hz]")
noise_w = float_slider(value=0.3, min=0.0, max=5.0, step=0.05,␣
↪description="noise sigma")
nperseg_w = dropdown(options=[64, 128, 256, 512, 1024], value=64,␣
↪description='nperseg')
window_l_w = dropdown(
    options=["rectangular", "hann", "hamming", "blackman", "tukey", "triang"],
    value="hann",
    description="window")


info_psd = widgets.HTML()

fig_psd, ax_psd = plt.subplots(1, 2, figsize=(8, 3.6), sharey='all',␣
↪sharex='all')
amp_line, = ax_psd[0].plot([], [])
psd_line, = ax_psd[1].plot([], [])

ax_psd[0].set_xlabel("Frequency [Hz]")
ax_psd[0].set_ylabel("Amplitude**2")
ax_psd[0].set_title("Amplitude spectrum")

ax_psd[1].set_xlabel("Frequency [Hz]")
ax_psd[1].set_ylabel("PSD [signal$^2$/Hz]")
ax_psd[1].set_title("Welch PSD estimate")

def _update_psd(change=None):
    update_psd_plot(
        amp_line, psd_line, ax_psd[0], ax_psd[1], info_psd,
        nperseg=nperseg_w.value, window=window_l_w.value,
        fs=fs_fixed, N=N_w.value, f0=f0_p_w.value, noise_sigma=noise_w.value
    )
    fig_psd.canvas.draw_idle()

for w in (f0_p_w, noise_w, nperseg_w, window_l_w, N_w):
    w.observe(_update_psd, names="value")
_update_psd()

display(widgets.VBox([widgets.HBox([f0_p_w, noise_w]),
                      widgets.HBox([N_w, nperseg_w, window_l_w]),
                      info_psd, fig_psd.canvas]))
```

```
1.0
```

```
VBox(children=(HBox(children=(FloatSlider(value=10.0, continuous_update=False,␣
↪description='f0 [Hz]', layout=L…
```



1.4.1 Interpretation

Both plots use the same underlying signal, but they normalize the frequency content differently.

- In the **amplitude spectrum**, a narrow tone appears as a prominent line amplitude.
- In the **PSD**, broadband noise becomes easier to compare because the vertical axis has units of power per Hz.

For noisy measurements, the PSD is usually the better tool for identifying the noise floor.

1.5 Summary

In this notebook you saw that:

1. the FFT works on a discrete frequency grid with spacing $\Delta f = f_s/N$,
2. off-grid frequencies lead to spectral leakage,
3. windowing changes the trade-off between sidelobe suppression and peak width,
4. amplitude spectra and PSDs serve different purposes.

The next notebook can build on this with time-frequency analysis and digital filtering.

1.6 Widget 5. Spectrograms

Change nperseg and noverlap and observe how the spectrogram changes. Which settings make the transient burst easiest to localize in time? Which settings make its frequency easiest to identify?

[6]:
```
import numpy as np
import matplotlib.pyplot as plt
import ipywidgets as widgets
from scipy.signal import spectrogram, get_window



# ------------------------------------------------------------
```

```python
# Widget controls
# ------------------------------------------------------------
# assumes fs_fixed already exists in your notebook
# e.g. fs_fixed = 100.0

N_spec_w = widgets.Dropdown(
    options=[100, 500, 1000],
    value=1000,
    description="N samples",
)

f0_spec_w = widgets.FloatSlider(
    value=10.0,
    min=9.0,
    max=11.0,
    step=0.1,
    description="f0 [Hz]",
    continuous_update=False,
)

noise_spec_w = widgets.FloatSlider(
    value=0.3,
    min=0.0,
    max=5.0,
    step=0.05,
    description="noise sigma",
    continuous_update=False,
)

nperseg_spec_w = widgets.Dropdown(
    options=[64, 128, 256, 512],
    value=128,
    description="nperseg",
)

noverlap_spec_w = widgets.Dropdown(
    options=[0, 32, 64, 96, 128, 192, 256, 384],
    value=64,
    description="noverlap",
)

window_spec_w = widgets.Dropdown(
    options=["rectangular", "hann", "hamming", "blackman", "tukey", "triang"],
    value="hann",
    description="window",
)
```

```python
info_spec = widgets.HTML()

# ------------------------------------------------------------
# Figure
# ------------------------------------------------------------
fig_spec, ax_spec = plt.subplots(2, 1, figsize=(8, 5.4))
plt.close(fig_spec)  # prevent duplicate static display in some notebook setups

def _update_spectrogram(change=None):
    # Keep overlap valid when nperseg changes
    if noverlap_spec_w.value >= nperseg_spec_w.value:
        valid_overlaps = [v for v in noverlap_spec_w.options if v <␣
 ↪nperseg_spec_w.value]
        noverlap_spec_w.value = valid_overlaps[-1]

    update_spectrogram_plot(
        ax_time=ax_spec[0],
        ax_spec=ax_spec[1],
        info_html=info_spec,
        fs=fs_fixed,
        N=N_spec_w.value,
        f0=f0_spec_w.value,
        noise_sigma=noise_spec_w.value,
        nperseg=nperseg_spec_w.value,
        noverlap=noverlap_spec_w.value,
        window=window_spec_w.value,
    )
    fig_spec.canvas.draw_idle()

for w in (N_spec_w, f0_spec_w, noise_spec_w, nperseg_spec_w, noverlap_spec_w,␣
 ↪window_spec_w):
    w.observe(_update_spectrogram, names="value")

_update_spectrogram()

display(
    widgets.VBox([
        widgets.HBox([f0_spec_w, noise_spec_w]),
        widgets.HBox([N_spec_w, nperseg_spec_w, noverlap_spec_w,␣
 ↪window_spec_w]),
        info_spec,
        fig_spec.canvas,
    ])
)
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
```

```
Cell In[6], line 89
     84     w.observe(_update_spectrogram, names="value")
     86 _update_spectrogram()
     88 display(
---> 89     widgets.VBox([
     90         widgets.HBox([f0_spec_w, noise_spec_w]),
     91␣
 ↪        widgets.HBox([N_spec_w, nperseg_spec_w, noverlap_spec_w, window_spec_ ]),
     92         info_spec,
     93         fig_spec.canvas,
     94     ])
     95 )

File /opt/conda/lib/python3.13/site-packages/ipywidgets/widgets/widget_box.py:
 ↪64, in Box.__init__(self, children, **kwargs)
     62 def __init__(self, children=(), **kwargs):
     63     kwargs['children'] = children
---> 64     super().__init__(**kwargs)

File /opt/conda/lib/python3.13/site-packages/ipywidgets/widgets/widget.py:506,␣
 ↪in Widget.__init__(self, **kwargs)
    503 super().__init__(**kwargs)
    505 Widget._call_widget_constructed(self)
--> 506 self.open()

File /opt/conda/lib/python3.13/site-packages/ipywidgets/widgets/widget.py:525,␣
 ↪in Widget.open(self)
    523 """Open a comm to the frontend if one isn't already open."""
    524 if self.comm is None:
--> 525     state, buffer_paths, buffers = _remove_buffers(self.get_state())
    527     args = dict(target_name='jupyter.widget',
    528                 data={'state': state, 'buffer_paths': buffer_paths},
    529                 buffers=buffers,
    530                 metadata={'version': __protocol_version__}
    531                 )
    532     if self._model_id is not None:

File /opt/conda/lib/python3.13/site-packages/ipywidgets/widgets/widget.py:615,␣
 ↪in Widget.get_state(self, key, drop_defaults)
    613 for k in keys:
    614     to_json = self.trait_metadata(k, 'to_json', self._trait_to_json)
--> 615     value = to_json(getattr(self, k), self)
    616     if not drop_defaults or not self._compare(value, traits[k].
 ↪default_value):
    617         state[k] = value

File /opt/conda/lib/python3.13/site-packages/ipywidgets/widgets/widget.py:52, i␣
 ↪_widget_to_json(x, obj)
```

```
     50     return {k: _widget_to_json(v, obj) for k, v in x.items()}
     51 elif isinstance(x, (list, tuple)):
---> 52     return [_widget_to_json(v, obj) for v in x]
     53 elif isinstance(x, Widget):
     54     return "IPY_MODEL_" + x.model_id

File /opt/conda/lib/python3.13/site-packages/ipywidgets/widgets/widget.py:54, i␣
 ↪_widget_to_json(x, obj)
     52     return [_widget_to_json(v, obj) for v in x]
     53 elif isinstance(x, Widget):
---> 54     return "IPY_MODEL_" + x.model_id
     55 else:
     56     return x

File /opt/conda/lib/python3.13/site-packages/ipywidgets/widgets/widget.py:552,␣
 ↪in Widget.model_id(self)
    547 @property
    548 def model_id(self):
    549     """Gets the model id of this widget.
    550
    551     If a Comm doesn't exist yet, a Comm will be created automagically." "
--> 552     return self.comm.comm_id

AttributeError: 'NoneType' object has no attribute 'comm_id'
```

[ ]:

```
```